

Competitive Programming

Section 35 Editorial: Math Mixed Contest

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	3
2	A. Blocked Path	4
2.1	Algorithm	4
2.2	Correctness proof	4
2.3	Full C++ solution	4
2.4	Full Python solution	5
3	B. Coprime Pairs	7
3.1	Algorithm	7
3.2	Correctness proof	7
3.3	Full Python solution	7
3.4	Full C++ solution	8
4	C. At Least One	10
4.1	Algorithm	10
4.2	Correctness proof	10
4.3	Full C++ solution	10
4.4	Full Python solution	11
5	Testing discussion	12
6	What to carry forward	13

1 Editorial overview

Problem	Main tools
A. Blocked Path	combinations, subtract bad paths
B. Coprime Pairs	SPF, distinct prime factors, inclusion-exclusion
C. At Least One	complement probability

2 A. Blocked Path

2.1 Algorithm

```
total = C(H+W-2, H-1)
bad = C(R+C-2, R-1) * C(H-R+W-C, H-R)
answer = total - bad
```

2.2 Correctness proof

Every monotone path is determined by the positions of its down moves. The bad paths are exactly those that can be split into a monotone path from the start to the blocked cell and then from the blocked cell to the finish. These two parts are independent, so multiplication counts them. Subtracting all bad paths leaves exactly the valid paths. $\square$

2.3 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
    long long r = 1;
    while (b) {
        if (b & 1LL) r = r * a % MOD;
        a = a * a % MOD;
        b >>= 1LL;
    }
    return r;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int h, w, r, c;
    if (!(cin >> h >> w >> r >> c)) return 0;
    int limit = h + w;
    vector<long long> fact(limit + 1, 1), invfact(limit + 1, 1);
    for (int i = 1; i <= limit; i++) fact[i] = fact[i - 1] * i % MOD;
    invfact[limit] = mod_pow(fact[limit], MOD - 2);
```

```

for (int i = limit; i >= 1; i--) invfact[i - 1] = invfact[i] * i % MOD;
auto C = [&](int n, int k) -> long long {
    if (k < 0 || k > n) return 0;
    return fact[n] * invfact[k] % MOD * invfact[n - k] % MOD;
};
long long total = C(h + w - 2, h - 1);
long long through = C(r + c - 2, r - 1) * C(h - r + w - c, h - r) % MOD;
cout << (total - through + MOD) % MOD << '\n';
return 0;
}

```

2.4 Full Python solution

```

import sys

MOD = 1_000_000_007

def prepare(n: int) -> tuple[list[int], list[int]]:
    fact = [1] * (n + 1)
    for i in range(1, n + 1):
        fact[i] = fact[i - 1] * i % MOD
    invfact = [1] * (n + 1)
    invfact[n] = pow(fact[n], MOD - 2, MOD)
    for i in range(n, 0, -1):
        invfact[i - 1] = invfact[i] * i % MOD
    return fact, invfact

def comb(n: int, k: int, fact: list[int], invfact: list[int]) -> int:
    if k < 0 or k > n:
        return 0
    return fact[n] * invfact[k] % MOD * invfact[n - k] % MOD

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    h, w, r, c = data
    fact, invfact = prepare(h + w)
    total = comb(h + w - 2, h - 1, fact, invfact)
    through = (
        comb(r + c - 2, r - 1, fact, invfact)
        * comb(h - r + w - c, h - r, fact, invfact)
        % MOD
    )

```

```
    print((total - through) % MOD)

if __name__ == "__main__":
    main()
```

3 B. Coprime Pairs

3.1 Algorithm

Process numbers left to right. For current x , get its distinct prime factors. Use inclusion-exclusion over nonempty subsets to count previous numbers sharing at least one factor. Add $\text{previous_count} - \text{shared}$ to the answer, then update all subset-product counters for x .

3.2 Correctness proof

For current x , a previous number is non-coprime with x iff it is divisible by at least one distinct prime factor of x . The subset-product counters give the size of every intersection of these divisibility events. Inclusion-exclusion therefore gives the number of previous non-coprime values. All other previous values form coprime pairs with x . Each pair is counted when its second endpoint is processed. $\square$

3.3 Full Python solution

```
import sys
from collections import defaultdict

def spf_table(n: int) -> list[int]:
    spf = list(range(n + 1))
    if n >= 1:
        spf[1] = 1
    for p in range(2, n + 1):
        if spf[p] == p and p * p <= n:
            for x in range(p * p, n + 1, p):
                if spf[x] == x:
                    spf[x] = p
    return spf

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n = data[0]
    arr = data[1 : 1 + n]
    spf = spf_table(max(arr))
    seen = defaultdict(int)
    answer = 0
```

```

for idx, x in enumerate(arr):
    factors = []
    y = x
    while y > 1:
        p = spf[y]
        factors.append(p)
        while y % p == 0:
            y //= p
    share = 0
    for mask in range(1, 1 << len(factors)):
        prod = 1
        bits = 0
        for i, p in enumerate(factors):
            if (mask >> i) & 1:
                prod *= p
                bits += 1
        share += seen[prod] if bits % 2 else -seen[prod]
    answer += idx - share
    for mask in range(1, 1 << len(factors)):
        prod = 1
        for i, p in enumerate(factors):
            if (mask >> i) & 1:
                prod *= p
        seen[prod] += 1
print(answer)

```

3.4 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    vector<int> a(n);
    int mx = 1;
    for (int &x : a) {
        cin >> x;
        mx = max(mx, x);
    }

    vector<int> spf(mx + 1);
    iota(spf.begin(), spf.end(), 0);
    if (mx >= 1) spf[1] = 1;
    for (long long p = 2; p <= mx; p++) {

```

```

        if (spf[p] == p && p * p <= mx) {
            for (long long x = p * p; x <= mx; x += p) {
                if (spf[x] == x) spf[x] = p;
            }
        }
    }

    vector<int> seen(mx + 1, 0);
    long long answer = 0;
    for (int idx = 0; idx < n; idx++) {
        int x = a[idx];
        vector<int> factors;
        while (x > 1) {
            int p = spf[x];
            factors.push_back(p);
            while (x % p == 0) x /= p;
        }

        long long share = 0;
        int m = (int)factors.size();
        for (int mask = 1; mask < (1 << m); mask++) {
            int prod = 1, bits = 0;
            for (int i = 0; i < m; i++) {
                if (mask & (1 << i)) {
                    prod *= factors[i];
                    bits++;
                }
            }
            if (bits % 2) share += seen[prod];
            else share -= seen[prod];
        }
        answer += idx - share;

        for (int mask = 1; mask < (1 << m); mask++) {
            int prod = 1;
            for (int i = 0; i < m; i++) {
                if (mask & (1 << i)) prod *= factors[i];
            }
            seen[prod]++;
        }
    }

    cout << answer << '\n';
    return 0;
}

```

4 C. At Least One

4.1 Algorithm

Compute the probability that no event happens, then subtract from 1:

```
none = product((q_i-p_i)/q_i)
answer = 1 - none
```

4.2 Correctness proof

The complement of “at least one event happens” is “no event happens.” Events are independent, so the probability that none happen is the product of the individual failure probabilities. Complementing gives the desired probability. $\square$

4.3 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
    long long r = 1;
    while (b) {
        if (b & 1LL) r = r * a % MOD;
        a = a * a % MOD;
        b >>= 1LL;
    }
    return r;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    long long none = 1;
    for (int i = 0; i < n; i++) {
        long long p, q;
        cin >> p >> q;
        none = none * ((q - p) % MOD) % MOD * mod_pow(q, MOD - 2) % MOD;
    }
}
```

```

}
cout << (1 - none + MOD) % MOD << '\n';
return 0;
}

```

4.4 Full Python solution

```

import sys

MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    idx = 1
    none = 1
    for _ in range(n):
        p, q = data[idx], data[idx + 1]
        idx += 2
        none = none * ((q - p) % MOD) % MOD * pow(q, MOD - 2, MOD) % MOD
    print((1 - none) % MOD)

if __name__ == "__main__":
    main()

```

5 Testing discussion

The mixed contest generators compare against direct combinatorics, brute-force gcd pair checks, and exact rational probability. This phase is especially prone to off-by-one and complement mistakes, so stress tests are part of the workflow.

6 What to carry forward

Math contest problems are often a composition of small tools. The main skill is recognizing the composition early: combinations plus subtraction, sieve plus inclusion-exclusion, or probability plus complement.